Now compare it to this:

```
$handle = fopen('https://github.com', 'r');
$text = '';
while (!feof($handle)) {
    $text .= fgets($handle);
}
var_dump(memory_get_usage());
```

```
Output:
int(567592)
```

In the second example we're adding each line returned from `fgets()` to the `$text` variable, using string concatenation via the `.=` operator. `$text` accumulates (in memory) the bytes from the stream and significantly increases usage, as seen in the output.

Remote file interaction with `fopen()` is not limited to `http`. There are wrappers for many other protocols, which can be used as the `scheme://` portion of a URL. They are as follows:

Table 1-8. *URL schemes*

`file://`	Explicitly access local filesystem
`http://` `https://`	Web server access
`ftp://`	File Transfer Protocol
`php://<stream>`	Various I/O stream types (see below)
`zlib://`	zlib-based compression streams (requires PHP built with `zlib` support)
`data://`	RFC 2397 – data is included as part of the URL
`glob://`	Find paths matching a pattern
`phar://`	PHP Archive file format
`ssh2://`	Secure Shell v.2 (requires `SSH2` extension)
`rar://`	RAR archive access (requires `rar` extension)
`ogg://`	Compressed audio (OGG/Vorbis codec, requires `OGG/Vorbis` extension)
`expect://`	Access other processes via STDIN/STDOUT/STDERR (requires `Expect` extension)